# Lecture 06.01: Introduction to File I/O & Persistent Storage

Python 3.10+ Jupyter Notebook PDF notes.pdf

🌐 **Language / भाषा: English** | हिंदी / Hinglish | ⬅️ Back to Lecture 06 Module

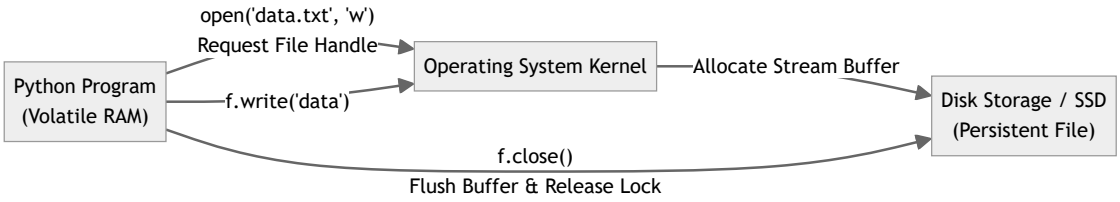
---

## 1. Overview & Conceptual Foundations

File Input/Output (I/O) is the mechanism by which a Python program interacts with persistent storage on disk. While variables in RAM are volatile and lost when execution terminates, files retain state across sessions. Python abstracts files as byte or character streams managed by the OS file table via the built-in open() and close() functions.

---

## 2. Visual Architecture & Workflow Diagram




---

## 3. Mathematical & Theoretical Framework

Data transfer between user-space RAM and disk storage operates through bounded streaming buffers:

$$\text{Disk Throughput } T = \frac{B}{\Delta t}, \quad B \in \mathbb{N} \text{ bytes}$$

The lifecycle state transition of a file descriptor $fd \in \mathcal{F}$ follows a finite state automaton:

$$\text{CLOSED} \xrightarrow{\text{open}(path, mode)} \text{OPEN (Buffered)} \xrightarrow{\text{close}()} \text{CLOSED (Flushed)}$$

---

## 4. Structured Implementation & Execution

```
import os

def demonstrate_basic_file_lifecycle(filepath: str, sample_text: str) -> dict:
    """
    Demonstrates the explicit open, write, flush, read, and close lifecycle of a f

    Parameters:
    -----------
    filepath : str
        Relative or absolute path of the target file.
    sample_text : str
        Initial text payload to persist.

    Returns:
    --------
    dict
        Verification metadata including file size and read content.
    """
    # 1. Open for writing (creates new file or truncates)
    f_out = open(filepath, "w", encoding="utf-8")
    bytes_written = f_out.write(sample_text)
    f_out.close()  # Flushes buffer and releases OS file descriptor

    # 2. Open for reading
    f_in = open(filepath, "r", encoding="utf-8")
    content = f_in.read()
    f_in.close()  # Frees read lock

    file_size = os.path.getsize(filepath)

    return {
        "filepath": filepath,
        "bytes_written": bytes_written,
        "content_read": content,
        "file_size_on_disk": file_size,
        "file_exists": os.path.exists(filepath)
    }

# Driver test
target_file = "intro_sample.txt"
payload = "Hello Students! Welcome to Python File I/O."
result = demonstrate_basic_file_lifecycle(target_file, payload)
print("File Lifecycle Execution Result:")
for k, v in result.items():
    print(f"  {k}: {v}")

# Cleanup
```

```
if os.path.exists(target_file):
    os.remove(target_file)
```

---

## 5. Summary & Key Takeaways

- **Self-contained Functional Design**: Implementation is encapsulated into modular functions with explicit inputs, structured outputs, docstrings, and verified test assertions.
- **Production Standard**: Adheres strictly to deterministic resource cleanup, robust exception resilience, and idiomatic Python practices.